

1. Two Sum

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`

Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`

Output: `[0,1]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$
- **Only one valid answer exists.**

The screenshot displays the LeetCode interface for the 'Two Sum' problem. On the left, the problem description is shown, including the input/output examples and constraints. The main area on the right contains a Python code editor with the following solution:

```
1 class Solution:
2     def twoSum(self, nums: List[int], target: int) -> List[int]:
3         prevMap = {} # val:index
4
5         for i, n in enumerate(nums):
6             diff = target - n
7             if diff in prevMap:
8                 return [prevMap[diff], i]
9             prevMap[n] = i
10
11     return
```

Below the code editor, the 'Testcase' and 'Test Result' sections are visible. The 'Test Result' shows 'Accepted' with a runtime of 0 ms. The input is `nums = [2,7,11,15]` and `target = 9`. The output is `[0,1]`.

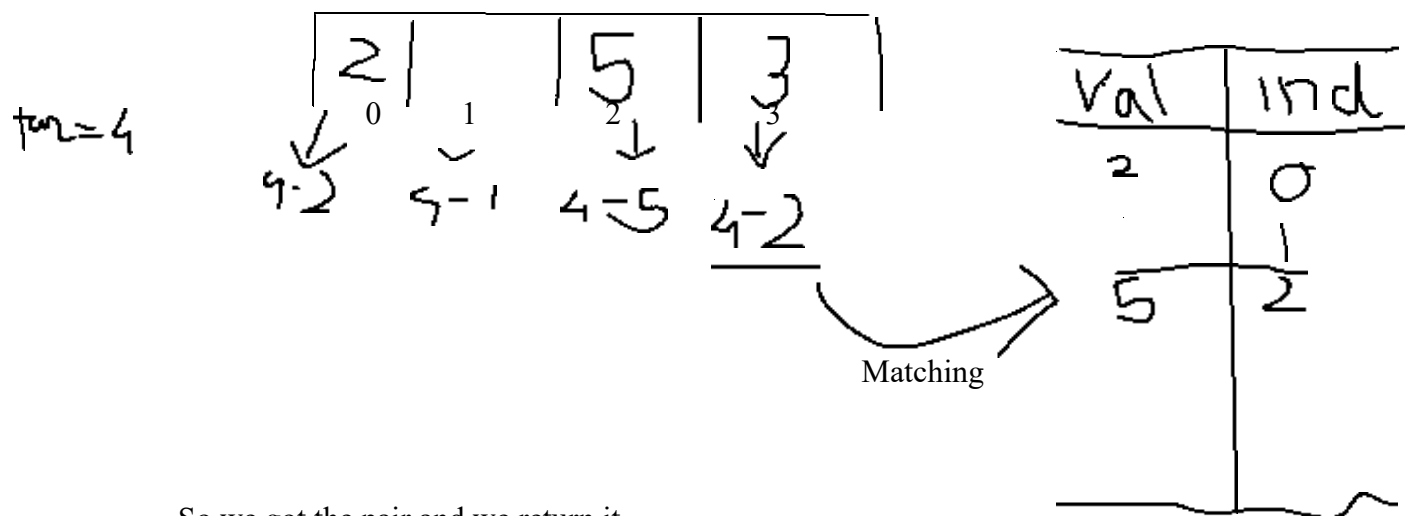
Solution:

Approch 1 To understand take an array and apply brute force first so let's consider array [2,7,11,15] target = 9

So compare each element with each element to n element takes n traverse so the thing is this one is having the complexity as $O(n^2)$

Approch 2 : we have to find a optimal way for this so we can use a hashmap as we all know it stores the values and indexes

So let us provide a hashmap



So we got the pair and we return it

So let's explain the code

So as we can see we have given

class Solution:

```
def twoSum(self, nums: List[int], target: int) -> List[int]:  
    # so this is given to us and we have to perform our operations so a list is  
    # created and a target is set to so first is create a hashmap  
    prevMap = {} # val: index so this is our map  
  
    for i, n in enumerate(nums):  
        diff = target - n  
        if diff in prevMap:  
            return [prevMap[diff], i]  
        prevMap[n] = i  
    return
```